

Objective:

To demonstrate NumberFormatException

Execution Steps:

STEP 1:

Open IntelliJ IDEA.

STEP 2:

Create a Java project.

STEP 3:

Create a package named:

javacore

STEP 4:

Create a Java class named:

NumberFormatException

STEP 5:

Write the given program in the class.

STEP 6:

Save the program.

STEP 7:

Compile the program.

STEP 8:

Run the program.

STEP 9:

The program starts execution from the main() method.

STEP 10:

The statement

```
String value = "100kg";
```

stores "100kg" in the variable value.

STEP 11:

The try block starts executing.

STEP 12:

Integer.parseInt(value) tries to convert "100kg" into an integer.

STEP 13:

"100kg" is not a valid integer because it contains "kg".

STEP 14:

NumberFormatException occurs.

STEP 15:

The catch block handles the exception.

STEP 16:

"Number Format Exception" is displayed.

STEP 17:

"Program ended" is displayed.

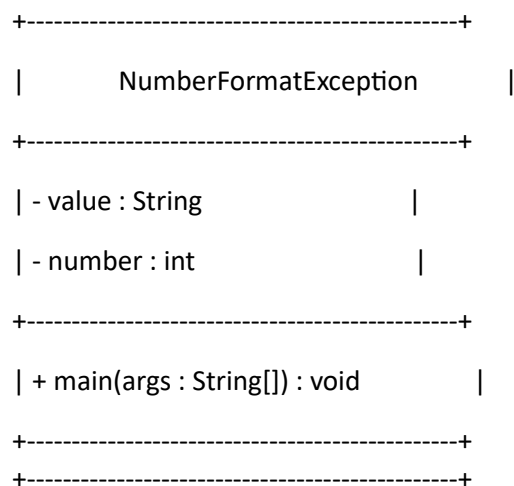
STEP 18:

The program ends.

Algorithm:

1. Start the program.
2. Create a String variable named value and store "100kg".
3. Start the try block.
4. Convert the String value into an integer using Integer.parseInt().
5. Since "100kg" contains letters, it cannot be converted into an integer.
6. NumberFormatException occurs.
7. Catch the exception using the catch block.
8. Display "Number Format Exception".
9. Display "Program ended".
10. Stop the program.

UML Diagram:



```

|      NumberFormatException      |
+-----+
| - value : String                |
| - number : int                  |
+-----+
| + main(args : String[]) : void  |
+-----+
|
|   handles
|   v
+-----+
|      NumberFormatException <<Exception>>      |
+-----+
|
|
+-----+
| + NumberFormatException()          |
+-----+

```

Output:

Number Format Exception

Program ended